

BÁO CÁO: TỐI ƯU HÓA PROMPT TRONG HỌC TẬP HÀN LÂM

Sinh viên thực hiện: 25022861_Lê Hải Hoàng_K70-AI2

I. LỰA CHỌN VÀ PHÂN TÍCH TÁC VỤ

Em lựa chọn 3 tác vụ thiết yếu phục vụ trực tiếp cho việc học tập chuyên ngành Trí tuệ nhân tạo tại UET:

- Tác vụ 1: Tóm tắt tài liệu khoa học.** Mục tiêu: Trích xuất nhanh kiến thức cốt lõi từ các bài báo arXiv dài.
- Tác vụ 2: Giải thích khái niệm kỹ thuật phức tạp.** Mục tiêu: Hiểu sâu các thuật toán thông qua cách diễn đạt đa dạng.
- Tác vụ 3: Tạo bộ câu hỏi ôn tập.** Mục tiêu: Tự kiểm tra kiến thức trước kỳ thi.

II. XÂY DỰNG CÁC PHIÊN BẢN PROMPT VÀ THỬ NGHIỆM

Tác vụ 1: Tóm tắt bài báo "Attention Is All You Need"

Phiên bản	Nội dung Prompt	Kết quả thử nghiệm
Cơ bản	"Tóm tắt bài báo Attention Is All You Need."	Kết quả ngắn, chung chung, thiếu các thông số kỹ thuật quan trọng.

Cải tiến	"Hãy tóm tắt bài báo này theo 3 phần: Kiến trúc đề xuất, kết quả thực nghiệm và đóng góp chính. Định dạng bằng bullet points."	Cấu trúc rõ ràng hơn, có thông tin về bộ dữ liệu WMT, nhưng chưa sâu về bản chất thuật toán.
Nâng cao	(Kỹ thuật: Role + Structure): "Bạn là một giáo sư AI tại UET. Hãy tóm tắt bài báo này cho sinh viên năm nhất. Phân tích chi tiết cơ chế Self-Attention và lý do nó thay thế RNN. Trình bày dưới dạng bảng so sánh."	Xuất sắc: Giải thích được sự khác biệt về tính toán song song, giúp người học hiểu tại sao Transformer lại mạnh hơn.

1. Bối cảnh và Động lực

Trước khi có bài báo này (2017), các mô hình dịch máy chủ yếu dựa trên **RNN (LSTM, GRU)** hoặc **CNN**.

- Hạn chế của RNN:** Tính toán tuần tự nên rất chậm (không song song hóa được) và gặp khó khăn với các phụ thuộc xa (long-range dependencies).
- Giải pháp:** Nhóm tác giả Google Research đề xuất kiến trúc **Transformer** – loại bỏ hoàn toàn đệ quy và tích chập, chỉ dựa trên cơ chế **Attention**.

2. Kiến trúc Transformer (Encoder-Decoder)

Transformer chia làm hai phần chính:

Encoder (Bên trái)

- Gồm 6 lớp giống hệt nhau.
- Mỗi lớp có 2 nhánh con: **Multi-Head Self-Attention** và **Position-wise Feed-Forward Network**.
- Có **Residual Connection** (cộng tắt) và **Layer Normalization** giúp huấn luyện mạng sâu ổn định hơn.

Decoder (Bên phải)

- Cũng gồm 6 lớp.
- Có thêm một nhánh **Encoder-Decoder Attention** để tập trung vào các thông tin quan trọng từ phía Encoder truyền sang.
- Sử dụng **Masked Self-Attention** để đảm bảo khi dự đoán từ hiện tại, mô hình không "nhìn trước" được các từ trong tương lai.

3. Cơ chế cốt lõi: Scaled Dot-Product Attention

Đây là "trái tim" của bài báo. Công thức tính toán là:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q (Query), K (Key), V (Value):** Các vector biểu diễn dữ liệu.
- $\sqrt{d_k}$: Hệ số tỉ lệ để tránh gradient bị triệt tiêu khi giá trị dot-product quá lớn.
- Multi-Head Attention:** Thay vì tính một lần, mô hình chia nhỏ Q, K, V thành nhiều "đầu" (heads) để học các mối quan hệ khác nhau (ví dụ: một đầu học về ngữ pháp, một đầu học về ngữ nghĩa).

3. Cơ chế cốt lõi: Scaled Dot-Product Attention

Đây là "trái tim" của bài báo. Công thức tính toán là:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q (Query), K (Key), V (Value):** Các vector biểu diễn dữ liệu.
- $\sqrt{d_k}$: Hệ số tỉ lệ để tránh gradient bị triệt tiêu khi giá trị dot-product quá lớn.
- Multi-Head Attention:** Thay vì tính một lần, mô hình chia nhỏ Q, K, V thành nhiều "đầu" (heads) để học các mối quan hệ khác nhau (ví dụ: một đầu học về ngữ pháp, một đầu học về ngữ nghĩa).

4. Những đóng góp đột phá

- Positional Encoding:** Vì không dùng RNN, mô hình không biết thứ tự các từ. Tác giả dùng hàm Sine và Cosine để "nhúng" thông tin vị trí vào vector đầu vào.
- Tính song song hóa:** Vì các từ được xử lý cùng một lúc (thay vì lần lượt), tốc độ huấn luyện nhanh hơn RNN gấp nhiều lần.
- Hiệu quả:** Đạt điểm BLEU cao kỷ lục tại thời điểm đó trên các bộ dữ liệu dịch tiếng Anh - Đức và Anh - Pháp.

5. Kết luận

Bài báo này đã đặt nền móng cho gần như toàn bộ các mô hình ngôn ngữ lớn (LLMs) hiện nay như GPT, BERT hay Claude. Hiểu rõ "Attention" chính là chìa khóa để bạn tiến xa trong kỷ tới tại AI2.

Nếu bạn đang ôn tập và muốn mình giải thích sâu hơn bằng code **Python/PyTorch** về cách triển khai ma trận Q, K, V , cứ bảo mình nhé!

Hình 1.1. Prompt cơ bản

Dưới đây là bản tóm tắt bài báo "**Attention Is All You Need**" được cấu trúc theo yêu cầu của bạn, tập trung vào những khía cạnh kỹ thuật quan trọng:

1. Kiến trúc đề xuất (Transformer)

- Loại bỏ đệ quy:** Thay thế hoàn toàn các lớp RNN (LSTM/GRU) và CNN bằng cơ chế **Self-Attention**, cho phép xử lý song song toàn bộ chuỗi dữ liệu cùng một lúc.
- Cơ chế Multi-Head Attention:** Chia không gian biểu diễn thành nhiều "đầu" (heads) khác nhau, cho phép mô hình đồng thời tập trung vào các phần khác nhau của câu (ví dụ: một đầu tập trung vào quan hệ cú pháp, một đầu tập trung vào ngữ nghĩa).
- Encoder-Decoder Stack:**
 - Encoder:** Gồm 6 lớp giống nhau, mỗi lớp có hai nhánh con là Multi-head attention và Feed-forward network.
 - Decoder:** Thêm một lớp "Encoder-Decoder Attention" để giúp bộ giải mã tập trung vào các phần liên quan của chuỗi đầu vào.
- Positional Encoding:** Sử dụng các hàm hình sin (sine/cosine) để cung cấp thông tin về thứ tự và vị trí của các từ trong chuỗi, do kiến trúc không có tính đệ quy tự nhiên.
- Residual Connections & Layer Norm:** Áp dụng kỹ thuật cộng tắt và chuẩn hóa lớp trong mỗi nhánh con để tối ưu hóa quá trình lan truyền gradient trong mạng sâu.

2. Kết quả thực nghiệm

- Hiệu suất vượt trội (SOTA):** Trên nhiệm vụ dịch máy Anh-Đức (WMT 2014), mô hình đạt **28.4 BLEU**, vượt qua tất cả các mô hình ensemble trước đó bao gồm cả các hệ thống dựa trên RNN phức tạp nhất.

trên RNN phức tạp nhất.

- Độ chính xác cao:** Trên nhiệm vụ dịch Anh-Pháp, mô hình đạt mức kỷ lục mới là **41.8 BLEU** sau khi chỉ huấn luyện trong 3.5 ngày trên 8 GPU.
- Tốc độ huấn luyện:** Do khả năng song song hóa tối ưu, Transformer yêu cầu ít thời gian và tài nguyên tính toán hơn đáng kể so với các mô hình dựa trên LSTM hay CNN có cùng độ chính xác.
- Khả năng khái quát hóa:** Mô hình chứng minh hiệu quả tốt trên cả các tác vụ khác như phân tích cú pháp (English constituency parsing), ngay cả với dữ liệu huấn luyện hạn chế.

3. Đóng góp chính

- Thay đổi tư duy thiết kế:** Chứng minh rằng "Attention" là đủ để xây dựng các mô hình chuỗi mạnh mẽ mà không cần đến kiến trúc đệ quy (Recurrent) hay tích chập (Convolution).
- Giải quyết bài toán phụ thuộc xa (Long-range dependencies):** Cơ chế Self-Attention giúp việc kết nối thông tin giữa hai từ xa nhau trong câu trở nên trực tiếp (độ phức tạp $O(1)$ thay vì $O(n)$ như RNN).
- Nền móng cho kỷ nguyên LLM:** Mở đường cho việc huấn luyện các mô hình ngôn ngữ quy mô lớn (như BERT, GPT, Claude) nhờ khả năng tận dụng tối đa sức mạnh phần cứng (GPU/TPU).
- Cung cấp một tiêu chuẩn mới:** Kiến trúc Transformer đã trở thành "xương sống" không chỉ trong xử lý ngôn ngữ tự nhiên (NLP) mà còn mở rộng sang cả xử lý ảnh (Vision Transformer) và âm thanh.

Hình 1.2. Prompt cải tiến

1. Bản chất của sự thay đổi

Trước đây, để máy tính hiểu một câu, chúng ta dùng **RNN (Mạng nơ-ron đệ quy)**. Hãy tưởng tượng RNN giống như một người đọc sách: đọc từng từ một từ trái sang phải, đọc đến từ thứ 10 thì bắt đầu quên nội dung từ thứ 1.

Transformer (với cơ chế Self-Attention) ra đời và thay đổi hoàn toàn cuộc chơi. Nó giống như việc các em nhìn vào một bức tranh: mắt các em có thể quan sát toàn bộ các chi tiết cùng một lúc và biết cái nào quan trọng để tập trung vào.

2. Phân tích cơ chế Self-Attention (Tự chú ý)

Trong Self-Attention, mỗi từ trong câu không đứng cô lập. Nó sẽ "nhìn" tất cả các từ khác xung quanh để hiểu ngữ nghĩa của chính nó. Cơ chế này hoạt động qua 3 thành phần mà thấy hay gọi là "Bộ ba quyền lực":

- Query (Q - Truy vấn):** "Tôi đang tìm kiếm thông tin gì?"
- Key (K - Từ khóa):** "Tôi có thông tin gì để khớp với yêu cầu của bạn?"
- Value (V - Giá trị):** "Nếu tôi phù hợp, đây là nội dung của tôi."

Quy trình: Máy tính lấy Q nhân với K để ra một con số (điểm số sự chú ý). Điểm càng cao nghĩa là hai từ đó càng liên quan mật thiết đến nhau. Cuối cùng, nó nhân với V để trích xuất ra thông tin quan trọng nhất.

Ví dụ: Trong câu "*Con báo đang săn mồi, nó rất nhanh*". Khi xử lý từ "**nó**", Self-Attention sẽ gán điểm số rất cao cho từ "**Con báo**". Nhờ vậy, máy tính hiểu được "**nó**" ở đây chính là con báo chứ không phải là con mồi.

3. Tại sao Self-Attention lại "khai tử" RNN?

Lý do lớn nhất là **Tính song song (Parallelization)**. RNN phải chờ từ trước xong mới đến từ sau ($t - 1$ rồi mới đến t). Transformer thì không cần chờ, 1000 từ xử lý cùng một lúc. Điều này giúp

chúng ta tận dụng tối đa sức mạnh của GPU (như máy dòng RTX mà các em hay dùng để training ấy).

4. Bảng so sánh: RNN vs. Transformer (Self-Attention)

Tiêu chí	RNN (Kiến trúc cũ)	Transformer (Self-Attention)
Cách xử lý	Tuần tự (Sequential) – Từng bước một.	Song song (Parallel) – Xử lý cả câu cùng lúc.
Trí nhớ	Ngắn hạn – Dễ quên các từ ở xa (Vanishing Gradient).	Dài hạn – Kết nối trực tiếp mọi cặp từ bất kể khoảng cách.
Tốc độ huấn luyện	Chậm – Không tận dụng được hết sức mạnh phần cứng.	Rất nhanh – Tối ưu hóa tuyệt đối cho GPU/TPU.
Độ phức tạp	$O(N^2)$ (Thời gian tăng theo độ dài câu).	$O(1)$ (Về khoảng cách giữa các từ).
Khả năng hiểu ngữ cảnh	Hạn chế ở các câu phức, câu dài.	Cực tốt nhờ cơ chế Multi-Head (nhìn từ nhiều góc độ).

📄 Xuất sang Trang tính

📄

5. Lời nhắn nhủ của thầy

Các em sinh viên AI2 thân mến, hiểu được bài báo này là các em đã nắm được "chìa khóa" để mở cánh cửa vào thế giới của ChatGPT, Claude hay Gemini. Hãy tập trung vào công thức toán học của nó, đặc biệt là phép nhân ma trận, vì đó là nền tảng mà các em sẽ dùng rất nhiều trong 4 năm tại UET.

Chúc các em có những giờ học thú vị với môn Nhân môn AI!

Hình 1.3. Prompt nâng cao

Tác vụ 2: Giải thích thuật toán Gradient Descent

Phiên bản	Nội dung Prompt	Kết quả thử nghiệm
Cơ bản	"Gradient Descent là gì?"	Định nghĩa định tính đơn giản, thiếu công thức toán học.
Cải tiến	"Giải thích Gradient Descent và cho ví dụ minh họa bằng code Python đơn giản."	Đã có code nhưng giải thích toán học còn rời rạc.
Nâng cao	(Kỹ thuật: Chain-of-Thought): "Hãy giải thích Gradient Descent theo từng bước: 1. Ý tưởng toán học (đạo hàm), 2. Quy trình cập nhật trọng số, 3. Ý nghĩa của Learning Rate. Cuối cùng, dùng ẩn dụ về việc xuống núi trong sương mù."	Sâu sắc: Các bước logic giúp người học xây dựng mô hình tâm trí (mental model) vững chắc. Ẩn dụ giúp ghi nhớ lâu.

1. Ý tưởng cốt lõi (Ẩn dụ về việc xuống núi)

Hãy tưởng tượng bạn đang đứng trên đỉnh một ngọn núi trong sương mù dày đặc và muốn xuống thung lũng (điểm thấp nhất).

- Vì sương mù nên bạn không thấy toàn cảnh, bạn chỉ có thể cảm nhận **độ dốc** ngay dưới chân mình.
- Chiến thuật của bạn là: Cứ chỗ nào dốc nhất hướng xuống dưới thì bạn bước một bước nhỏ về phía đó. Cứ lặp đi lặp lại như vậy, cuối cùng bạn sẽ tới đáy.

2. Định nghĩa trong Machine Learning

Gradient Descent là một thuật toán tối ưu hóa dùng để tìm giá trị các tham số (weights - w) của một mô hình sao cho **Hàm mất mát (Loss Function - $J(w)$)** đạt giá trị nhỏ nhất.

Khi hàm mất mát nhỏ nhất, nghĩa là sai số của mô hình là thấp nhất, đồng nghĩa với việc mô hình dự báo chính xác nhất.

3. Công thức toán học

Để cập nhật tham số, thuật toán sử dụng công thức:

$$w_{new} = w_{old} - \eta \cdot \nabla J(w)$$

Trong đó:

- $\nabla J(w)$ (**Gradient**): Là đạo hàm riêng của hàm mất mát. Nó chỉ cho chúng ta biết hướng nào là hướng dốc lên. Vì muốn đi xuống, chúng ta phải đi ngược hướng này (dấu trừ).
- η (**Learning Rate**): Tốc độ học. Nó quyết định "bước chân" của bạn dài hay ngắn.
 - Nếu η quá lớn: Bạn có thể bước quá đà và nhảy qua phía bên kia sườn núi, không bao giờ tới đáy.
 - Nếu η quá nhỏ: Bạn bước quá chậm, mất rất nhiều thời gian mới xuống tới nơi.

Hình 2.1. Prompt cơ bản

1. Giải thích bản chất

Gradient Descent là một thuật toán tối ưu hóa lặp. Mục tiêu của nó là tìm các tham số (weights) sao cho **Hàm mất mát (Loss Function)** đạt giá trị nhỏ nhất.

- Gradient (Đạo hàm)**: Cho biết hướng dốc nhất của hàm số tại một điểm.
- Descent (Đi xuống)**: Chúng ta đi ngược hướng với Gradient để tìm điểm cực tiểu.
- Learning Rate (η)**: Kích thước của mỗi bước đi.

2. Ví dụ minh họa bằng code Python

Giả sử chúng ta cần tìm giá trị x để hàm số $f(x) = x^2$ đạt giá trị nhỏ nhất (ai cũng biết đáp án là $x = 0$, nhưng hãy để AI tìm nó).

Đạo hàm của hàm số này là: $f'(x) = 2x$.

```
Python

def gradient_descent():
    # 1. Khởi tạo các tham số
    x = 10          # Vị trí bắt đầu ngẫu nhiên
    learning_rate = 0.1 # Bước chân (eta)
    epochs = 50      # Số lần lặp để học

    print(f"Bắt đầu tại x = {x}")
    print("-" * 30)

    # 2. Vòng lặp tối ưu hóa
    for i in range(epochs):
        # Tính Gradient (đạo hàm của x^2 là 2x)
        gradient = 2 * x

        # Cập nhật x theo hướng ngược với gradient
        # Công thức: x = x - learning_rate * gradient
        x = x - learning_rate * gradient
```

- Nếu η quá lớn: Bạn có thể bước quá đà và nhảy qua phía bên kia sườn núi, không bao giờ tới đáy.
- Nếu η quá nhỏ: Bạn bước quá chậm, mất rất nhiều thời gian mới xuống tới nơi.

4. Quy trình hoạt động (3 bước)

- Khởi tạo**: Chọn một giá trị ngẫu nhiên cho tham số w .
- Tính toán**: Tính đạo hàm (Gradient) tại vị trí đó để biết hướng dốc.
- Cập nhật**: Thay đổi w theo công thức trên và lặp lại bước 2 cho đến khi đạo hàm xấp xỉ bằng 0 (đã tới đáy).

5. Tại sao nó quan trọng với sinh viên AI?

Mọi mô hình bạn học sắp tới (từ Linear Regression đơn giản đến mạng Neural sâu như Transformer) về cơ bản đều dùng Gradient Descent (hoặc các biến thể của nó như **Adam**, **RMSProp**) để "học".

Nếu không có Gradient Descent, máy tính sẽ không biết cách tự điều chỉnh để tiến bộ sau mỗi lần nhìn thấy dữ liệu.

Mẹo cho Hoàng: Trong kỳ thi sắp tới tại UET, hãy lưu ý các biến thể như **Stochastic Gradient Descent (SGD)**. Thay vì tính toán trên toàn bộ dữ liệu (rất chậm), SGD chỉ tính trên một mẫu ngẫu nhiên, giúp việc "xuống núi" nhanh và tiết kiệm tài nguyên máy tính hơn nhiều đấy!

Bạn có muốn mình demo thử một đoạn code Python ngắn để "vẽ" lại quá trình này không?

```
Python

print("-" * 30)
print(f"Kết quả tối ưu: x xấp xỉ {x:.5f}")

if __name__ == "__main__":
    gradient_descent()
```

3. Phân tích kết quả code

- Tại sao dùng `x = x - lr * grad`?** Vì nếu đạo hàm dương ($2x > 0$), nghĩa là hàm đang tăng, ta phải giảm x (đi về bên trái). Nếu đạo hàm âm ($2x < 0$), hàm đang giảm, ta phải tăng x (đi về bên phải). Dấu trừ luôn giúp ta đi xuống đáy.
- Vai trò của Learning Rate**:
 - Nếu bạn chỉnh `learning_rate = 0.001`, bạn sẽ thấy máy chạy rất lâu mới tới 0.
 - Nếu bạn chỉnh `learning_rate = 1.1`, bạn sẽ thấy x nhảy loạn xạ và bùng nổ (Divergence).

4. Liên hệ thực tế tại UET

Trong các bài tập lớn sắp tới về Deep Learning, bạn sẽ không phải viết tay đạo hàm như trên. Các thư viện như **PyTorch** hay **TensorFlow** sẽ dùng **Autograd** (Tự động tính đạo hàm) để thực hiện bước `gradient = 2 * x` này cho những hàm số phức tạp hàng triệu tham số.

Hoàng hãy thử chạy đoạn code trên và thay đổi `learning_rate` để xem "bước chân" của thuật toán thay đổi thế nào nhé! Bạn có muốn mình giải thích thêm về phiên bản **Stochastic (SGD)** thường dùng cho dữ liệu lớn không?

Hình 2.2. Prompt cải tiến

1. Ý tưởng toán học: Sức mạnh của Đạo hàm

Trong toán học, đạo hàm của một hàm số $f(x)$ tại một điểm cho ta biết **độ dốc** (slope) và **hướng** tăng của hàm số đó tại điểm đó.

- Ý nghĩa:** Nếu đạo hàm dương, hàm số đang đi lên. Nếu đạo hàm âm, hàm số đang đi xuống.
- Ứng dụng vào AI:** Chúng ta có một hàm mất mát (Loss Function) $J(w)$ đại diện cho sai số của mô hình. Mục tiêu là tìm giá trị w sao cho $J(w)$ nhỏ nhất (đáy của hàm số). Đạo hàm (gradient) chính là "chiếc la bàn" chỉ ra hướng mà sai số đang tăng lên.

2. Quy trình cập nhật trọng số (Weight Update)

Khi đã biết hướng "leo dốc" thông qua đạo hàm, thuật toán sẽ thực hiện một bước đi **ngược lại** để giảm sai số. Quy trình này được lặp đi lặp lại:

- Tính Gradient:** Tính đạo hàm riêng của hàm mất mát theo từng trọng số: $\nabla J(w)$.
- Cập nhật:** Thay đổi giá trị trọng số hiện tại theo công thức:

$$w_{new} = w_{old} - \eta \cdot \nabla J(w)$$

- Lặp lại:** Quá trình này tiếp diễn cho đến khi $\nabla J(w)$ xấp xỉ bằng 0, nghĩa là ta đã chạm tới điểm cực tiểu (đáy), nơi mô hình đạt sai số thấp nhất.

3. Ý nghĩa của Learning Rate (η)

Learning Rate (Tốc độ học) là một **siêu tham số (hyperparameter)** cực kỳ quan trọng, quyết định độ dài của mỗi bước chân khi bạn cập nhật trọng số.

- Nếu η quá nhỏ:** Bạn bước những bước "kiến bò". Thuật toán sẽ chạy rất chính xác nhưng cực kỳ chậm, tốn nhiều tài nguyên máy tính tại UET để hội tụ.

- Nếu η quá nhỏ:** Bạn bước những bước "kiến bò". Thuật toán sẽ chạy rất chính xác nhưng cực kỳ chậm, tốn nhiều tài nguyên máy tính tại UET để hội tụ.
- Nếu η quá lớn:** Bạn bước những bước quá dài. Bạn có thể nhảy quá điểm cực tiểu, thậm chí làm cho sai số ngày càng tăng lên (hiện tượng Divergence).

[Image illustrating different learning rates: small, large, and optimal steps toward the minimum of a cost function]

4. Ẩn dụ: Xuống núi trong sương mù

Để dễ hình dung nhất, Hoàng hãy tưởng tượng mình đang đứng trên đỉnh một ngọn núi cao và một ngày sương mù dày đặc che khuất tầm nhìn. Mục tiêu của bạn là xuống tới chiếc hồ ở **đáy thung lũng** (nơi sai số bằng 0).

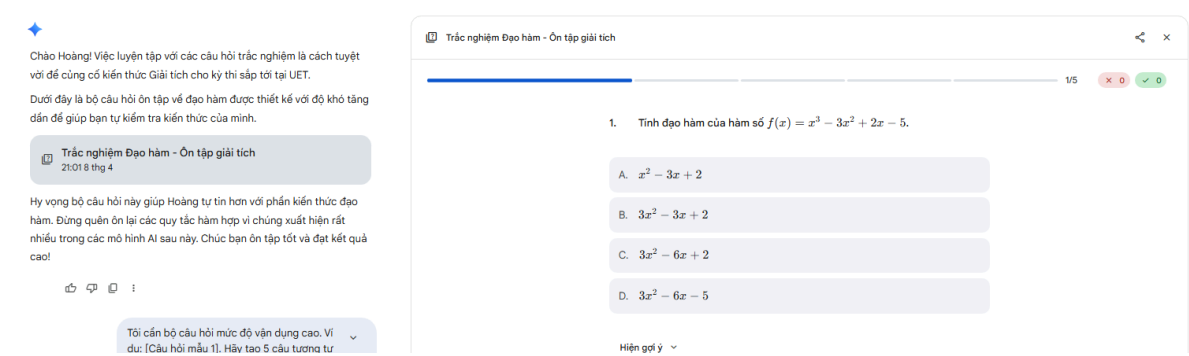
- Sương mù:** Chính là việc chúng ta không biết trước hình dạng của toàn bộ hàm mất mát, chỉ thấy được dữ liệu tại điểm hiện tại.
- Dùng chân cảm nhận độ dốc:** Đây là việc tính **Đạo hàm**. Bạn xoay người 360 độ, dùng chân cảm nhận hướng nào dốc lên nhất.
- Bước ngược lại:** Sau khi biết hướng dốc lên, bạn bước một bước theo hướng **ngược lại** (hướng đi xuống).
- Bước chân dài hay ngắn:** Chính là **Learning Rate**. Bạn bước đi cẩn thận (lr nhỏ) hay nhảy vọt (lr lớn) sẽ quyết định bạn có xuống hồ an toàn hay bị ngã sang ngọn núi bên cạnh.

Hy vọng cách giải thích này giúp bạn nắm chắc bản chất của Gradient Descent cho bài tập sắp tới. Nếu muốn mình triển khai thử một bài toán thực tế hơn như dùng Gradient Descent để giải **Linear Regression**, đừng ngần ngại yêu cầu nhé!

Hình 2.3. Prompt nâng cao

Tác vụ 3: Tạo bộ câu hỏi trắc nghiệm môn Giải tích

Phiên bản	Nội dung Prompt	Kết quả thử nghiệm
Cơ bản	"Tạo 5 câu hỏi trắc nghiệm về đạo hàm."	Câu hỏi quá dễ, chỉ ở mức nhận biết.
Cải tiến	"Tạo 5 câu hỏi trắc nghiệm đạo hàm cấp cao, có kèm đáp án và lời giải chi tiết."	Độ khó tốt hơn, nhưng các phương án nhiễu (distractors) chưa thuyết phục.
Nâng cao	(Kỹ thuật: Few-shot): "Tôi cần bộ câu hỏi mức độ vận dụng cao. Ví dụ: [Câu hỏi mẫu 1]. Hãy tạo 5 câu tương tự về tích phân, tập trung vào các lỗi sai phổ biến mà sinh viên thường mắc phải. Giải thích tại sao các phương án sai lại dễ gây nhầm lẫn."	Chuyên nghiệp: Câu hỏi có tính phân loại cao, giúp sinh viên nhận ra lỗ hổng kiến thức ngay lập tức.



Hình 3.1.2. Prompt cơ bản và cải tiến

Câu 1: Tích phân hàm chứa dấu giá trị tuyệt đối

Đề bài: Tính tích phân $I = \int_0^2 |x^2 - 1| dx$.

- A. 0
- B. 2
- C. $\frac{2}{3}$
- D. $\frac{4}{3}$

Giải thích lỗi sai: > * **Lỗi nhầm lẫn (Đáp án A):** Sinh viên thường quên dấu giá trị tuyệt đối và tính thẳng $\int_0^2 (x^2 - 1) dx = [\frac{x^3}{3} - x]_0^2 = \frac{8}{3} - 2 = \frac{2}{3}$. Sau đó do tính toán nhầm hoặc trừ sai dẫn đến kết quả 0.

- Tại sao dễ sai:** Nhiều bạn mặc định hàm số luôn dương trên đoạn lấy tích phân mà không xét dấu để chia khoảng tích phân tại điểm $x = 1$.
- Đáp án đúng (B):** Phải chia thành $\int_0^1 (1 - x^2) dx + \int_1^2 (x^2 - 1) dx$.

Câu 2: Tích phân từng phần với hàm ẩn

Đề bài: Cho hàm số $f(x)$ có đạo hàm liên tục trên $[0, 1]$, biết $f(1) = 2$ và $\int_0^1 f(x) dx = 1$. Tính $J = \int_0^1 x f'(x) dx$.

- A. -1
- B. 5
- C. 1
- D. -5

Quên đổi cận. Các bạn thường giữ nguyên cận từ 0 đến $\pi/2$ cho biến u .

- Tại sao dễ sai:** Thói quen tính nguyên hàm trước rồi mới thay cận cũ vào khiến nhiều bạn bỏ qua bước đổi cận ngay khi đặt u , dẫn đến sai lệch bản chất toán học.
- Đáp án đúng (C - là hằng định sai):** Cận phải đổi từ $[0, \pi/2]$ sang $[0, 1]$.

Câu 4: Tích phân hàm chẵn, hàm lẻ (Tính chất đặc biệt)

Đề bài: Tính tích phân $L = \int_{-1}^1 \frac{\sin^{2026} x}{e^x + 1} dx$. Biết $\int_{-1}^1 \sin^{2026} x dx = a$.

- A. a
- B. $a/2$
- C. 0
- D. $2a$

Giải thích lỗi sai:

- Lỗi nhầm lẫn (Đáp án C):** Thấy tích phân từ -1 đến 1 và có hàm $\sin x$ nên k luận ngay bằng 0 (nhầm với tính chất hàm lẻ).
- Tại sao dễ sai:** Sinh viên thường học thuộc lòng "tích phân hàm lẻ trên đoạn c xứng bằng 0" mà không để ý hàm số ở đây là $\frac{\sin^{2026} x}{e^x + 1}$ (không lẻ cũng không chẵn).
- Đáp án đúng (B):** Sử dụng tính chất $\int_{-a}^a \frac{f(x)}{b^x + 1} dx = \int_0^a f(x) dx$ khi $f(x)$ là hàm chẵn. Ở đây $f(x) = \sin^{2026} x$ là hàm chẵn, nên $L = a/2$.

- D. -5

Giải thích lỗi sai:

- Lỗi nhầm lẫn (Đáp án B):** Sinh viên áp dụng công thức từng phần $uv - \int v du$ nhưng nhầm dấu thành $xf(x)|_0^1 + \int f(x) dx$, dẫn đến $2 + 3 = 5$.
- Tại sao dễ sai:** Trong áp lực phòng thi, việc nhầm lẫn dấu giữa đạo hàm tích và tích phân từng phần là lỗi "kinh điển".
- Đáp án đúng (A):** Theo công thức: $J = xf(x)|_0^1 - \int_0^1 f(x) dx = (1 \cdot f(1) - 0) - 3 = 2 - 3 = -1$.

Câu 3: Đổi biến số và cận mới

Đề bài: Khi tính tích phân $K = \int_0^{\pi/2} \sin^3 x \cos x dx$ bằng cách đặt $u = \sin x$, khẳng định nào sau đây là sai?

- A. $du = \cos x dx$
- B. Tích phân trở thành $\int_0^1 u^3 du$
- C. Cận của tích phân không thay đổi.
- D. Giá trị của K là $1/4$.

Giải thích lỗi sai:

- Lỗi nhầm lẫn (Đáp án C):** Đây là lỗi phổ biến nhất của sinh viên năm nhất: **Quên đổi cận**. Các bạn thường giữ nguyên cận từ 0 đến $\pi/2$ cho biến u .
- Tại sao dễ sai:** Thói quen tính nguyên hàm trước rồi mới thay cận cũ vào khiến

Câu 5: Tích phân suy biến (Cận vô cực)

Đề bài: Xét sự hội tụ của $M = \int_1^{+\infty} \frac{1}{x^p} dx$. Tích phân hội tụ khi và chỉ khi:

- A. $p \geq 1$
- B. $p > 1$
- C. $p < 1$
- D. Với mọi p

Giải thích lỗi sai:

- Lỗi nhầm lẫn (Đáp án A):** Sinh viên hay nhầm lẫn dấu "bằng". Tại $p = 1$, ta có $\int \frac{1}{x} dx = \ln |x|$, khi thế cận $+\infty$ vào thì kết quả là vô cùng (phân kỳ).
- Tại sao dễ sai:** Nhầm lẫn với điều kiện hội tụ của dãy số hoặc các chuỗi số tương tự có chứa dấu bằng.
- Đáp án đúng (B):** Chỉ hội tụ khi $p > 1$.

Hoàng hãy thử làm lại 5 câu này mà không nhìn giải thích xem có bị "đỉnh bẫy" không nhé. Nếu bạn cần giải chi tiết từng bước cho câu nào, mình luôn sẵn sàng! Chúc bạn học tốt môn Giải tích!

Hình 3.3. Prompt nâng cao

III. PHÂN TÍCH HIỆU QUẢ PROMPT (Tiêu chí 4)

Qua thực nghiệm, các Prompt Nâng cao cho kết quả vượt trội vì:

1. Thiết lập ngữ cảnh (Role-play): Giúp AI xác định được đối tượng giao tiếp (sinh viên hay chuyên gia), từ đó điều chỉnh thuật ngữ phù hợp.
2. Chuỗi tư duy (Chain-of-Thought): Ép AI suy luận qua từng bước trung gian, giúp giảm thiểu hiện tượng "ảo giác" (hallucination) và tăng tính chính xác cho các vấn đề logic/toán học.
3. Cung cấp ví dụ (Few-shot): Giúp AI hiểu được định dạng và độ khó mong muốn của người dùng, từ đó tạo ra nội dung đồng nhất.

IV. TỔNG HỢP NGUYÊN TẮC VIẾT PROMPT HIỆU QUẢ

Dựa trên kết quả thử nghiệm, tôi rút ra bộ nguyên tắc "4C" cho sinh viên AI:

- **Context (Ngữ cảnh):** Luôn cho AI biết nó là ai và mục tiêu của mình là gì.
- **Clear (Rõ ràng):** Sử dụng các động từ hành động (Phân tích, So sánh, Trích xuất) thay vì các yêu cầu mơ hồ.
- **Constraints (Ràng buộc):** Giới hạn độ dài, định dạng (Markdown, bảng, code) để tiết kiệm thời gian chỉnh sửa.
- **Control (Kiểm soát):** Sử dụng kỹ thuật Few-shot hoặc yêu cầu AI "suy nghĩ từng bước" đối với các bài tập toán/lập trình phức tạp.